

Übungsblatt 8

Ü 8.1 Threads

Beantworten Sie die folgenden Fragen im Kontext von Threads.

- Was bedeutet „thread-safe“ und welche Eigenschaften sind damit verbunden? Erklären Sie in diesem Zusammenhang auch den Begriff „reentrant“.
- Welche Möglichkeiten gibt es, um Threads abubrechen (engl. cancel)? Beschreiben und diskutieren Sie diese Möglichkeiten.
- Was sind und wozu dienen Thread-Pools? Wie lassen sich diese mit POSIX-Threads implementieren?

Ü 8.2 Threads – Berechnung

Schreiben Sie ein multithreaded Programm in C, das verschiedene statistische Werte für eine Liste von Zahlen berechnet. Dieses Programm wird eine Reihe von Zahlen als Input annehmen (z.B. `int numbers[ARRAY_SIZE] = {22, 28, 3, 65, 43, 22, 12, 99, 102, 38, 21}`). Drei Worker-Threads erstellen dann drei statistische Indikatoren: Ein Thread berechnet den Durchschnitt, ein zweiter den maximalen Wert, ein dritter den minimalen Wert. Zum Beispiel nehmen wir an, Ihr Programm erhält die Zahlen:

{22, 28, 3, 65, 43, 22, 12, 99, 102, 38, 21}

Das Programm soll Folgendes ausgeben:

AVG: 41.36

MIN: 3

MAX: 102

Achtung! Das Programm soll sich sauber beenden lassen

Ü 8.3 Threads/Sync (Semaphoren)

Sie haben es geschafft. Studium abgeschlossen und Ihr erster Tag im neuen Job. Ein Kollege bittet Sie um Hilfe; er hat eine Semaphore verwendet und irgendwie *‘verkeilen’* sich die Threads manchmal. Der Kollege wirkt schon sehr verzweifelt, setzt aber alle Hoffnungen in Sie, weil Sie ja sowas studiert haben.

Das Programm finden Sie beim Übungsblatt (`sema.c`)

1. Suchen Sie das Problem in Code
2. Beheben Sie das Problem
3. Wie nennt man dieses Verhalten?

Ü 8.4 Bus Simulation Teil 1

Schreiben Sie ein C-Programm mit Threads, das den Transport von Passagieren in einem Bus simuliert. Jeder Passagier wird als eigener Thread realisiert und kann in den Bus ein- und aussteigen.

Während der Fahrt schlafen die Passagiere ein (verwenden Sie `passenger_sleep()`). Nachdem sie aufgewacht sind, steigen sie an der nächsten Haltestelle aus.

Nutzen Sie `bus_stub.c` als Grundlage. Sehen Sie sich auch die Header-Datei `visualization.h` an. Diese Datei und `visualization.c` können Sie unbearbeitet lassen; sie enthalten die Logik und eine grafische Darstellung des Busses, der zwischen den zwei Haltestellen pendelt. Diese Visualisierung wurde unter Ubuntu geschrieben und kann leider zu unerwartetem Verhalten auf anderen Distributionen führen.

1. Implementieren Sie das Erstellen und Aufräumen von Passagier-Threads (`pthread_create` und `pthread_join`).
2. Sichern Sie den Zugriff auf die geteilten Variablen (z.B. den Status der Passagiere) mit einem Mutex.
3. Nutzen Sie Condition Variables (`pthread_cond_t`), damit die Passagiere an der Haltestelle warten, bis der Bus ankommt.
4. Wenn der Bus da ist, steigen die Passagiere ein (Status `BUS_STANDING`).
5. Während der Fahrt schlafen die Passagiere (`passenger_sleep()`).
6. Danach warten sie, bis der Bus an der *nächsten* Haltestelle hält, steigen aus (Status `DONE`) und beenden ihren Thread.

Nutzen Sie `bus_stub.c` als Grundlage. Kompilieren Sie mit: `gcc bus_stub.c visualization.c -o bus -lnurses -pthread`

Ü 8.5 Bus Simulation Teil 2

Erweitern Sie Ihr Programm aus Teil 1 um Ressourcenmanagement. Setzen Sie in der `visualization.h` die Passagieranzahl auf `#define N 26`.

Der Bus hat nun eine strikte Gesamtkapazität von M (9), davon X (4) Sitzplätze, und muss N (26) Passagiere befördern.

1. **Kapazität:** Beim Einsteigen muss nun geprüft werden, ob noch Platz im Bus frei ist. Ist der Bus voll, müssen die Passagiere an der Haltestelle auf die nächste Fahrt warten!
2. **Sitzplätze:** Passagiere konkurrieren beim Einsteigen um die X Sitzplätze (Status `BUS_SEATED`). Sind alle Sitze belegt, müssen die restlichen Passagiere stehen (Status `BUS_STANDING`).
3. **Aussteigen:** Beim Aussteigen müssen die Zähler für belegte Plätze/Sitzplätze wieder freigegeben werden.